

Xuanhua Li - DATA-260 - HOMEWORK 3
Student ID: 019122974

SID4: 2974
PORT_BASE: 8274
PREFIX: s2974
SEED: 2974
VERIFY_SEED: 262974
DOMAIN_ID: 6

Assigned domain: Rental housing listings
Hardware: Apple MacBook Air, M4, 16 GB RAM
Embedding model: [sentence-transformers/all-MiniLM-L6-v2](#)
GitHub repository: <https://github.com/carrovo/data260-2974>
Git tag: hw3
Tagged commit hash: 945ad08a645c5443f35e504e2fa94a1f4f03111d

Part1

Objective:

Create a FastAPI application that implements a simple user authentication system for your specific domain. The application should include:

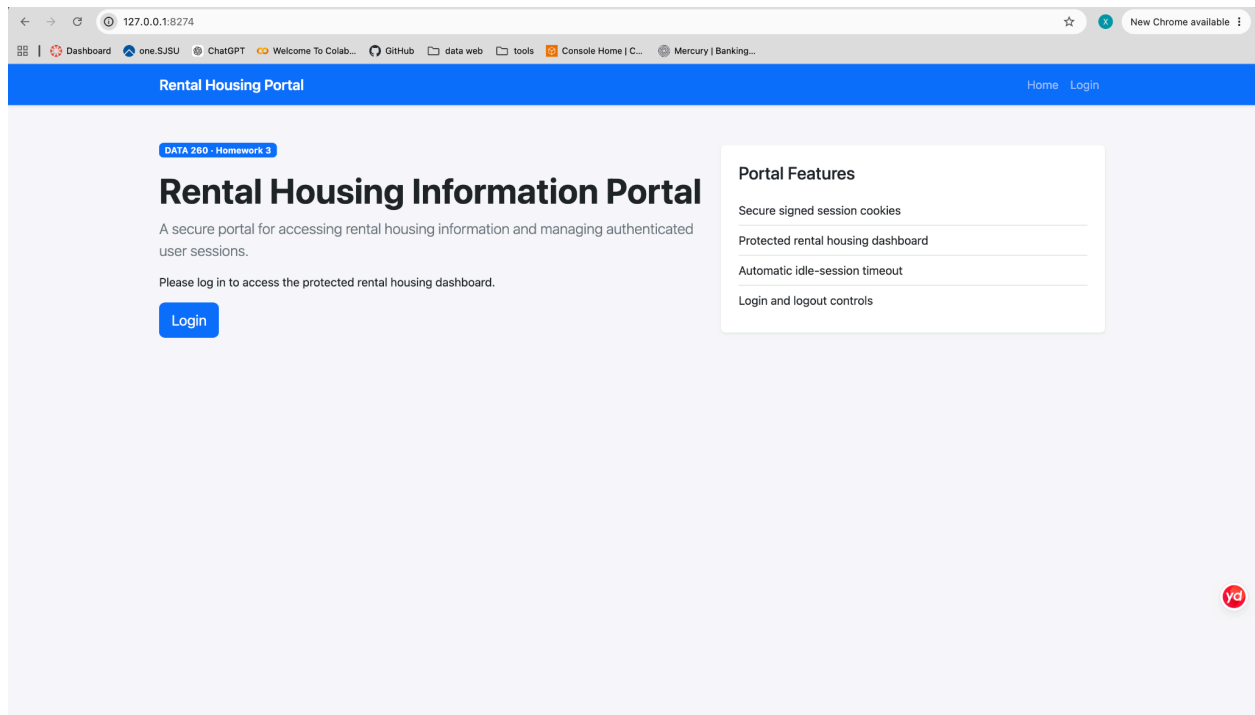
- User login and logout functionality
- Session management to keep users logged in
- Protected routes that only logged-in users can access
- Styling using Bootstrap to make the application visually appealing

Requirements:

1. Routes - Handled Separately in a router:

- Home Page (/): Display a welcome message for your domain application. Show a link to the login page if the user is not logged in. Show a link to the dashboard and log out if the user is logged in.

The home page displays the Rental Housing Listings domain and changes its navigation options based on the user's login status.



- Login Page (/login): Display a login form with fields for username and password. Validate the credentials and log the user in if they are correct. Redirect to the dashboard on successful login. Display a Bootstrap alert message on the login page when a user enters invalid credentials.

The login route validates the username and password. Valid credentials create a session and redirect the user to /dashboard. Invalid credentials return the login page with a Bootstrap alert.

login:

The screenshot shows a web browser at the address 127.0.0.1:8274/login. The page has a blue header with "Rental Housing Portal" and links for "Home" and "Login". The main content area features a "Portal Login" form with the instruction "Enter your account information to access the protected rental housing dashboard." The form includes a "Username" field with a blue border and a blue eye icon, a "Password" field, a blue "Login" button, and a blue link "Return to Home". A red "yol" logo is in the bottom right corner. At the bottom of the page, it says "DATA 260 HW3 · Domain 6: Rental Housing Listings".

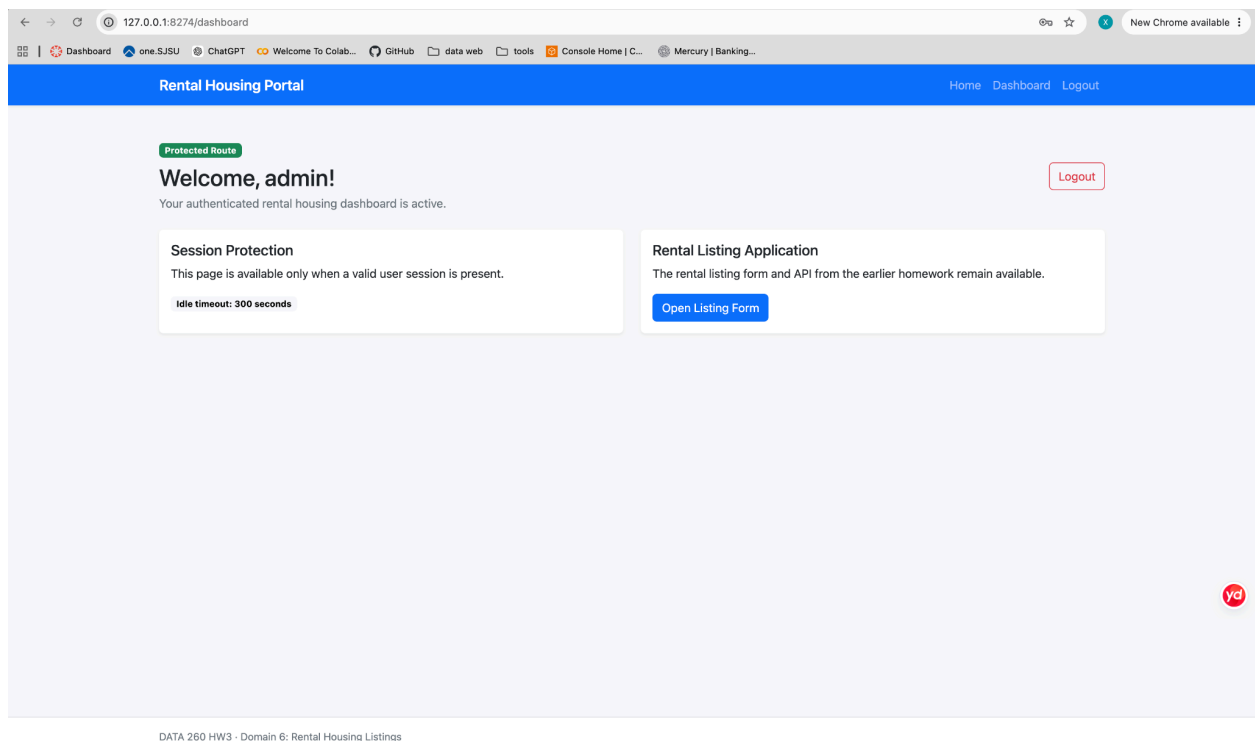
Wrong password invalid login:

This screenshot shows the same "Portal Login" form as the previous one, but with an error message. A red alert box at the top of the form contains the text "Invalid username or password." The "Username" field now contains the text "admin". The "Password" field is empty. The "Login" button and "Return to Home" link remain visible. The rest of the page, including the header, footer, and logo, is identical to the previous screenshot.

- Dashboard Page (/dashboard): Display a welcome message with the user's name. Show a logout link. Protect this route so only logged-in users can access it.
- Logout (/logout): Destroy the session and redirect the user to the home page.

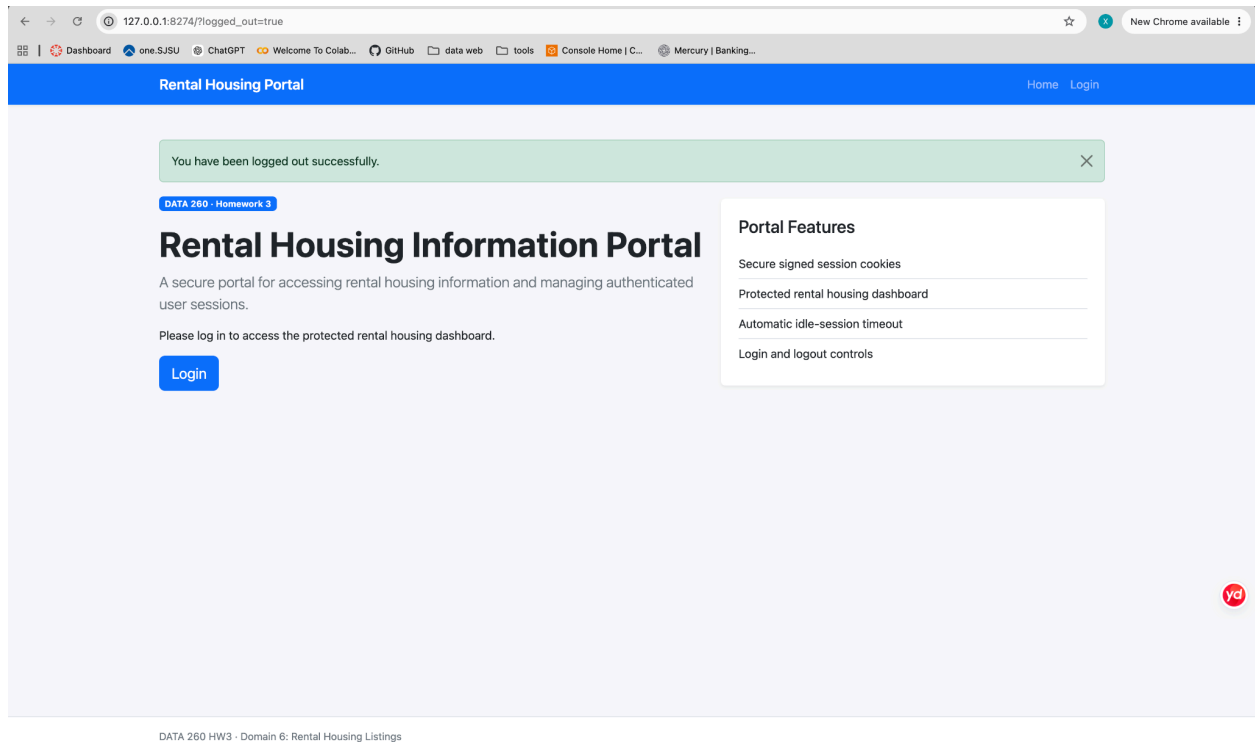
The dashboard checks the session before rendering. A user without a valid session is redirected to the login page.

Dashboard:



The logout route clears the session and redirects the user to the home page. The cleared session cannot be reused to access /dashboard.

Logout:



2. Session Management:

Use Starlette's SessionMiddleware to manage user sessions. Store the logged-in user's information in the session.

Ensure that the session cookie is secure. Capture and include the Set-Cookie response header showing all three attributes.

Implement an idle session timeout. Prove that an expired or logged-out session cannot be reused to reach /dashboard.

Cookie:

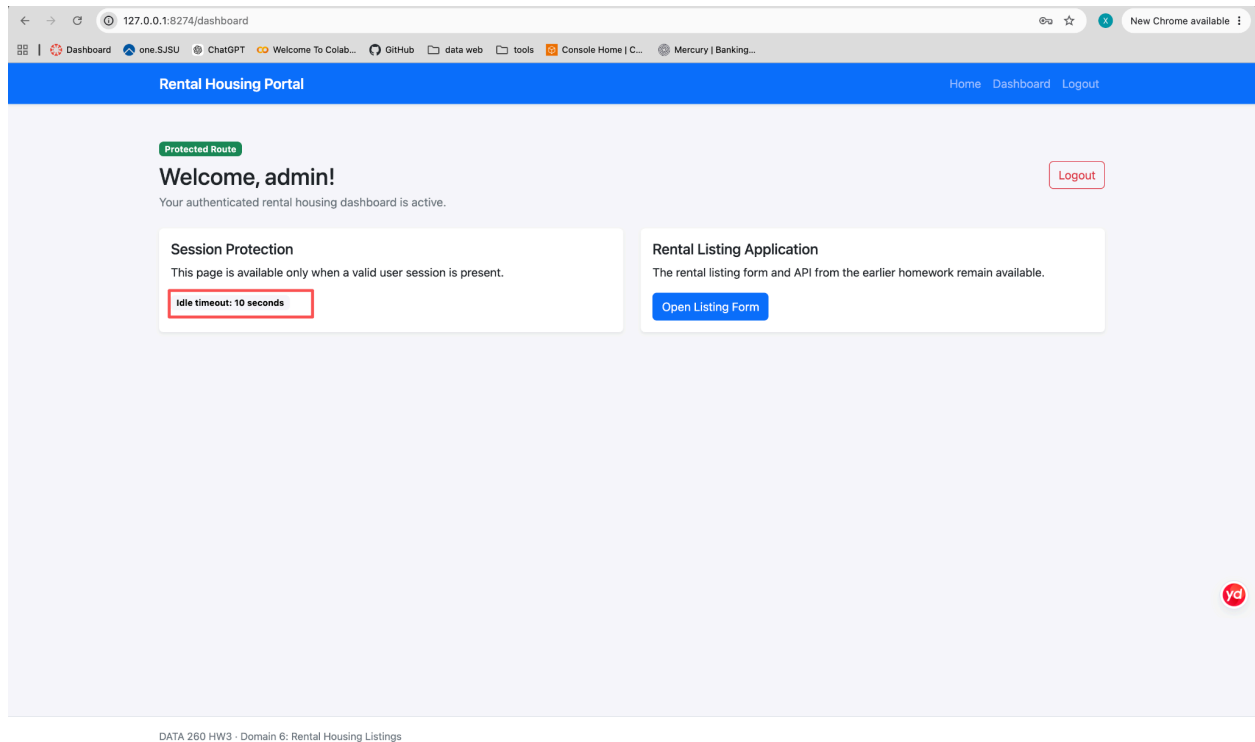
The Set-Cookie header contains HttpOnly, Secure, and SameSite=Lax. The cookie also has a maximum age of 3,600 seconds.

```

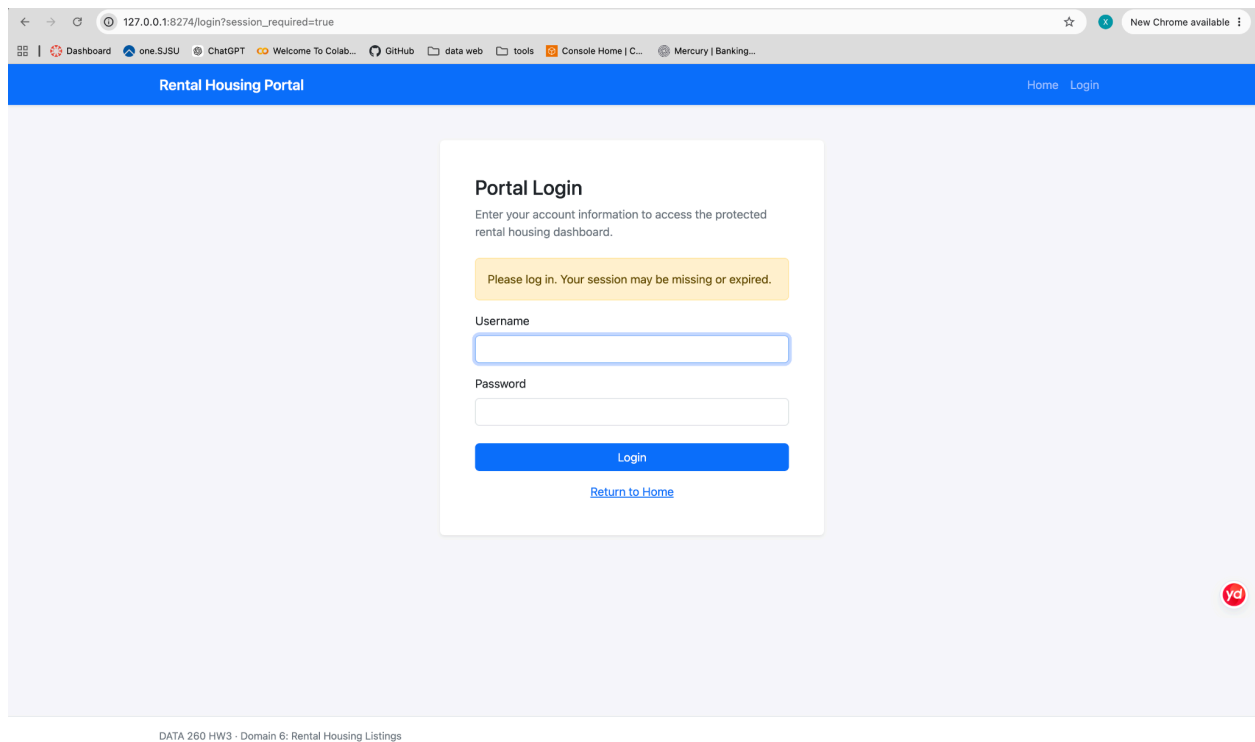
Problems  Output  Debug Console  Terminal  Ports
carol@Carols-MacBook-Air data260-2974 % curl -i \
-X POST \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin&password=password" \
http://127.0.0.1:8274/login
HTTP/1.1 303 See Other
date: Sun, 20 Sep 2026 23:54:02 GMT
server: uvicorn
content-length: 0
location: /dashboard
set-cookie: session=eyJ1c2VyIjogImFkbWluIiwgImxhc3RfYW0aXZpdHk1OiAxNzg5OTQ4NDQzLjI1Mjg3OTF9.arByGw.boxRISjv_Lh4sI0w5W3fFCuVweQ; path=/; Max-Age=3600; httponly; samesite=lax; secure
carol@Carols-MacBook-Air data260-2974 %

```

Timeout before:



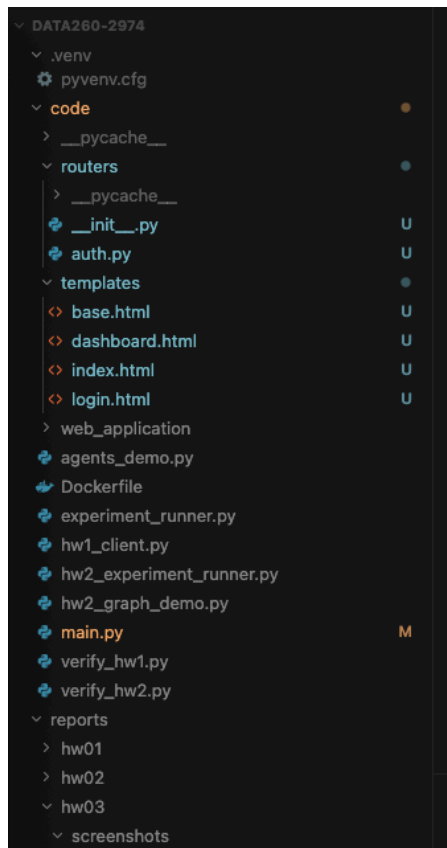
Timeout after:



For the timeout test, the idle timeout was temporarily reduced to 10 seconds. The first screenshot shows an active session. After the timeout period, the same session was redirected to the login page and could no longer access /dashboard.

3. Styling with Bootstrap: Explore and use Bootstrap to style all pages. Make the application responsive and visually appealing. Use Bootstrap components, such as the Navbar for navigation, Cards for forms and content, Buttons for actions, and Alerts for messages. Include one screenshot per route page. Include only the auth.py code at the bottom and one screenshot of your templates directory.

The application uses a shared Jinja base template and Bootstrap components, including a responsive Navbar, Cards, Buttons, Alerts, and the Bootstrap grid.



[auth.py](#):

```
import os
import secrets
import time
from pathlib import Path

from fastapi import APIRouter, Form, Request
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates

router = APIRouter() # Create a new API router.

TEMPLATES_DIR = Path(__file__).resolve().parents[1] / "templates"
```

```

templates = Jinja2Templates(directory=str(TEMPLATES_DIR)) # Create a new Jinja2
templates.

VALID_USERNAME = os.getenv("HW3_USERNAME", "admin") # Get the valid username from the
environment variables.
VALID_PASSWORD = os.getenv("HW3_PASSWORD", "password") # Get the valid password from
the environment variables.

IDLE_TIMEOUT_SECONDS = int(
    os.getenv("HW3_IDLE_TIMEOUT_SECONDS", "300")
)

def get_current_user(request: Request) -> str | None: # Get the current user.
    """
    Return the logged-in user when the session is valid.

    If the session has been idle for too long, clear it and
    treat the visitor as logged out.
    """
    user = request.session.get("user") # Get the user from the session.
    last_activity = request.session.get("last_activity") # Get the last activity from
the session.

    if not user or last_activity is None:
        return None

    current_time = time.time()
    idle_time = current_time - float(last_activity)

    if idle_time > IDLE_TIMEOUT_SECONDS: # Check if the idle time is greater than the
idle timeout seconds.
        request.session.clear() # Clear the session.
        return None

    request.session["last_activity"] = current_time # Set the last activity to the
current time.
    return user

@router.get("/")
def home(request: Request): # Get the home page.

```

```

    user = get_current_user(request) # Get the user from the session.

    return templates.TemplateResponse( # Return the home page.
        request=request,
        name="index.html",
        context={
            "user": user,
        },
    )

@router.get("/login")
def login_page(request: Request): # Get the login page.
    user = get_current_user(request)

    if user:
        return RedirectResponse(
            url="/dashboard",
            status_code=303,
        )

    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={
            "user": None,
            "error": None,
        },
    )

@router.post("/login")
def login( # Login.
    request: Request,
    username: str = Form(...),
    password: str = Form(...),
):
    username_is_valid = secrets.compare_digest(
        username,
        VALID_USERNAME,
    )

    password_is_valid = secrets.compare_digest(

```

```

        password,
        VALID_PASSWORD,
    )

    if not (username_is_valid and password_is_valid):
        request.session.clear()

    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={
            "user": None,
            "error": "Invalid username or password.",
            "entered_username": username,
        },
        status_code=401, # Return the login page with the error message.
    )

request.session.clear()
request.session["user"] = username
request.session["last_activity"] = time.time()

return RedirectResponse(
    url="/dashboard",
    status_code=303, # Redirect to the dashboard.
)

@router.get("/dashboard")
def dashboard(request: Request):
    user = get_current_user(request)

    if not user:
        return RedirectResponse(
            url="/login?session_required=true",
            status_code=303,
        )

    return templates.TemplateResponse(
        request=request,
        name="dashboard.html",
        context={

```

```
        "user": user,
        "idle_timeout_seconds": IDLE_TIMEOUT_SECONDS,
    },
)

@router.get("/logout")
def logout(request: Request):
    request.session.clear()

    return RedirectResponse(
        url="/?logged_out=true",
        status_code=303,
    )
```

Part2 Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG)

Dataset

Use Tiny Shakespeare (raw text) as a warm-up only:

<https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>

But your actual dataset (This will be the graded dataset) must be at least 200 KB of local snapshots of public documents relevant to your assigned domain (DOMAIN_ID). Store SOURCES.md with source URLs and access dates, plus CORPUS_MANIFEST.json with each local filename, byte size, and SHA-256 hash.

https://www.dre.ca.gov/publications/ResourceGuidebook/2026_Landlord_Tenant_Guide.pdf

https://calcivilrights.ca.gov/wp-content/uploads/sites/32/2022/12/Housing-Rights-Booklet_English.pdf

https://www.hud.gov/sites/documents/fheo_booklet_eng.pdf

Corpus file	Bytes
ca_fair_housing_rights_booklet.pdf	3,530,369
ca_landlord_tenant_guide_2026.pdf	3,952,345
hud_fair_housing_booklet.pdf	5,468,378
Total	12,951,092

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/check_corpus.py
ca_fair_housing_rights_booklet.pdf: pages=10, text_characters=11806
ca_landlord_tenant_guide_2026.pdf: pages=156, text_characters=458033
hud_fair_housing_booklet.pdf: pages=24, text_characters=26128

Files checked: 3
Total extracted characters: 495967
Corpus text extraction check passed.
```

The corpus contains three PDF files with a total size of 12,951,092 bytes, which exceeds the required 200 KB minimum. SOURCES.md records the source URLs and access dates. CORPUS_MANIFEST.json records the local filenames, sizes, and SHA-256 hashes.

A. Environment & Setup Install: llama-index, llama-index-embeddings-huggingface, sentence-transformers, faiss-cpu, numpy, pandas.

- Use a public sentence embedding model (e.g., sentence-transformers/all-MiniLM-L6-v2). (pull from huggingface)

- ```
sentence_window_splitter = (
 SentenceWindowNodeParser.from_defaults(
 window_size=SENTENCE_WINDOW_SIZE,
 window_metadata_key="window",
 original_text_metadata_key="original_text",
)
)
```

## 2. Indexing (in memory)

- Build a `VectorStoreIndex` over your nodes with an in-memory vector store (e.g., `SimpleVectorStore`) to keep everything local and fast. ( `LlamaIndex` )

- Build a `VectorStoreIndex` over your nodes with an in-memory vector store (e.g., `SimpleVectorStore`) to keep everything local and fast. ( `LlamaIndex` )

- Build a `VectorStoreIndex` over your nodes with an in-memory vector store (e.g., `SimpleVectorStore`) to keep everything local and fast. ( `LlamaIndex` )

```
def build_vector_index(
 nodes,
 embed_model: HuggingFaceEmbedding,
) -> VectorStoreIndex:
 vector_store = SimpleVectorStore()

 storage_context = StorageContext.from_defaults(
 vector_store=vector_store,
)

 index = VectorStoreIndex(
 nodes,
 storage_context=storage_context,
 embed_model=embed_model,
 show_progress=True,
)

 return index
```

Each technique used a separate in-memory SimpleVectorStore and VectorStoreIndex.

### 3. Retrieval-only function

Write a helper that, given a query and k, does the following:

- Compute the query embedding (show its dimension and the first 8 values).

```
query_embedding = embed_model.get_query_embedding(
 question_text
)
```

- Retrieve top-k nodes; for each, compute and print:
  - Store similarity score (if available from retriever).
  - Cosine similarity between the query embedding and the document embedding (compute embeddings of the returned chunks explicitly).
  - Chunk length and a short text preview (first ~160 chars).

```
retriever = index.as_retriever(
 similarity_top_k=TOP_K
)
```

```
cosine_similarity = (
 calculate_cosine_similarity(
 query_vector,
 document_vector,
)
)
```

```

preview = " ".join(
 retrieved_text.split()
)[:160]

store_score = node_with_score.score

result_rows.append(
 {
 "rank": rank,
 "store_score": (
 None
 if store_score is None
 else float(store_score)
),
 "cosine_sim": cosine_similarity,
 "chunk_len": len(indexed_text),
 "context_len": len(retrieved_text),
 "preview": preview,
 "source_file": source_file,
 "page_number": page_number,
 "source_match": (
 source_file == expected_source
),
 }
)

retrieval_latency_ms = (
 time.perf_counter() - retrieval_start
) * 1000

```

- Print the shapes of the query vector and the stacked doc vectors.

```

((.venv)) carol@Carols-MacBook-Air data260-2974 % tail -n 12 reports/hw03/RUN_LOG.txt
sentence_window q3 0.716897 0.685591 1 35.395375
sentence_window q4 0.555869 0.578528 1 35.947708
sentence_window q5 0.684296 0.694395 1 36.505666
token q1 0.726185 0.696028 1 10.127834
token q2 0.728498 0.703451 1 8.926375
token q3 0.727002 0.657718 1 9.782333
token q4 0.688493 0.633915 1 10.390958
token q5 0.763461 0.741050 1 9.625625

Saved summary CSV: /Users/carol/Documents/data260-2974/reports/hw03/raw/summary_metrics.csv
Saved summary JSON: /Users/carol/Documents/data260-2974/reports/hw03/raw/summary_metrics.json
Saved per-query CSV: /Users/carol/Documents/data260-2974/reports/hw03/raw/per_query_metrics.csv
((.venv)) carol@Carols-MacBook-Air data260-2974 % grep -E \
"Questions loaded|Readable PDF pages loaded|Query-technique runs completed|Formal retrieval experiment complete
d" \
reports/hw03/RUN_LOG.txt
Questions loaded: 5
Readable PDF pages loaded: 187
Query-technique runs completed: 15
Formal retrieval experiment completed.

```

Your printed output should clearly identify the technique used and list a table with: rank, store\_score, cosine\_sim, chunk\_len, preview. Save this per-query, per-technique output as machine-readable data (CSV/JSON/JSONL) in raw/, along with a script that recomputes the summary tables below from it.

### Query to use

Before running, create questions.yaml with five realistic domain questions (relevant to your DOMAIN\_ID). For each question, record an expected answer and expected source file; at least two questions must depend on information found in only one source document. Commit questions.yaml before adding the retrieval results so the expected answers are not rewritten after seeing the output.

| ID | Question                                                                                                                                         | Expected answer                                                                                               | Expected source file                               |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| q1 | What is the maximum security deposit a California landlord may generally charge, and what exception applies to qualifying small landlords?       | Generally one month's rent. A qualifying small landlord may charge up to two months' rent.                    | <a href="#">ca_landlord_tenant_guide_2026.pdf</a>  |
| q2 | How long does a California landlord have after a tenant moves out to return the security deposit or provide an itemized statement of deductions? | 21 calendar days or less.                                                                                     | <a href="#">ca_landlord_tenant_guide_2026.pdf</a>  |
| q3 | May a California housing provider charge a pet deposit or extra rent for an assistance animal needed because of a disability?                    | No. A housing provider may not charge a pet deposit or extra rent for a required assistance animal.           | <a href="#">ca_fair_housing_rights_booklet.pdf</a> |
| q4 | What must happen when a California residential lease is negotiated in a language other than English?                                             | For covered languages, the landlord must provide a written translation before the rental agreement is signed. | <a href="#">ca_fair_housing_rights_booklet.pdf</a> |

q5 How long after an alleged housing discrimination incident occurs or ends does a person have to file a complaint with HUD?

One year.

hud\_fair\_housing\_booklet.pdf

```
! questions.yaml
1 domain_id: 6
2 assigned_domain: "Rental housing listings"
3 question_count: 5
4
5 questions:
6 - id: "q1"
7 question: "What is the maximum security deposit a California landlord may generally charge, and what exception applies?"
8 expected_answer: "A California landlord may generally charge no more than one month's rent as a security deposit. A landlord may charge more if the tenant has a pet or a dog."
9 expected_source_file: "corpus/hw03/ca_landlord_tenant_guide_2026.pdf"
10
11 - id: "q2"
12 question: "How long does a California landlord have after a tenant moves out to return the security deposit or provide the required documentation?"
13 expected_answer: "The landlord has 21 days after the tenant moves out to return the deposit or provide the required documentation."
14 expected_source_file: "corpus/hw03/ca_landlord_tenant_guide_2026.pdf"
15
16 - id: "q3"
17 question: "May a California housing provider charge a pet deposit or extra rent for an assistance animal needed because of a disability?"
18 expected_answer: "No. A necessary assistance animal must be allowed as a reasonable accommodation without a pet deposit or extra rent."
19 expected_source_file: "corpus/hw03/ca_fair_housing_rights_booklet.pdf"
20
21 - id: "q4"
22 question: "What must happen when a California residential lease is negotiated in a language other than English?"
23 expected_answer: "The lease must be translated into the language used to negotiate it."
24 expected_source_file: "corpus/hw03/ca_fair_housing_rights_booklet.pdf"
25
26 - id: "q5"
27 question: "How long after an alleged housing discrimination incident occurs or ends does a person have to file a complaint with HUD?"
28 expected_answer: "A person has one year after the alleged discrimination occurred or ended to file a complaint with HUD."
29 expected_source_file: "corpus/hw03/hud_fair_housing_booklet.pdf"
```

The questions were selected before the formal retrieval experiment. Questions q1 and q2 depend on information from the California Landlord-Tenant Guide, while q5 depends on the HUD Fair Housing booklet. This satisfies the requirement that at least two questions rely on information found in one specific source document.

```
((.venv)) carol@Carols-MacBook-Air data260-2974 % git log -4 --oneline
912e9f5 (HEAD -> hw3) Add HW3 final verification
c37a7c8 Add HW3 retrieval experiment and metrics
e847548 Add HW3 retrieval environment and chunking warm-up
d2b3372 Add HW3 corpus and precommit evaluation questions
```

The Git history shows that questions.yaml was committed in d2b3372 before the formal retrieval experiment and its results were committed in c37a7c8.

### What to compare (report section)

After you run the three pipelines:

#### 1. Retrieval Quality:

- top-1 cosine (highest similarity among the top-k for that technique)
- mean@k cosine (average of top-k cosines)

- #chunks produced by the chunker and the avg chunk length (characters or tokens)
- retrieval latency in milliseconds (time the similarity search took; simple timer is fine)

```

cosine_values = [
 float(result["cosine_sim"])
 for result in results
]

top1_cosine = max(cosine_values)

mean_at_k_cosine = mean(
 cosine_values
)

recall_at_k = int(
 any(
 result["source_match"]
 for result in results
)
)

```

| Technique           | Chunks | Avg chunk length | Top-1 cosine | Mean@5 cosine | Recall@5 | Mean retrieval latency |
|---------------------|--------|------------------|--------------|---------------|----------|------------------------|
| Token               | 584    | 968.84           | 0.726728     | 0.686432      | 1.00     | 9.77 ms                |
| Semantic            | 441    | 1124.10          | 0.737317     | 0.671641      | 1.00     | 9.39 ms                |
| Sentence-win<br>dow | 3797   | 130.56           | 0.723145     | 0.651483      | 1.00     | 36.61 ms               |

```

((.venv)) carol@Carols-MacBook-Air data260-2974 % python -m py_compile code/hw3_compute_metrics.py
((.venv)) carol@Carols-MacBook-Air data260-2974 % python code/hw3_compute_metrics.py \
2>&1 | tee -a reports/hw03/RUN_LOG.txt
Summary metrics:
 technique chunks avg_chunk_length_chars top1_cosine mean_at_k_cosine recall_at_k mean_retrieval_latency_ms
 semantic 441 1124.095238 0.737317 0.671641 1 9.391292
sentence_window 3797 130.557282 0.723145 0.651483 1 36.609458
 token 584 968.840753 0.726728 0.686432 1 9.770625

Per-query metrics:
 technique query_id top1_cosine mean_at_k_cosine recall_at_k retrieval_latency_ms
 semantic q1 0.723798 0.652585 1 9.028333
 semantic q2 0.738375 0.698943 1 10.853375
 semantic q3 0.752428 0.662899 1 9.007167
 semantic q4 0.663607 0.603497 1 8.801750
 semantic q5 0.808375 0.740281 1 9.265833
sentence_window q1 0.742179 0.650316 1 40.895792
sentence_window q2 0.705494 0.648584 1 34.302750
sentence_window q3 0.748160 0.685591 1 35.395375
sentence_window q4 0.667865 0.578528 1 35.947708
sentence_window q5 0.752029 0.694395 1 36.505666
 token q1 0.726185 0.696028 1 10.127834
 token q2 0.728498 0.703451 1 8.926375
 token q3 0.727002 0.657718 1 9.782333
 token q4 0.688493 0.633915 1 10.390958
 token q5 0.763461 0.741050 1 9.625625

Saved summary CSV: /Users/carol/Documents/data260-2974/reports/hw03/raw/summary_metrics.csv
Saved summary JSON: /Users/carol/Documents/data260-2974/reports/hw03/raw/summary_metrics.json
Saved per-query CSV: /Users/carol/Documents/data260-2974/reports/hw03/raw/per_query_metrics.csv

```

All three techniques achieved a Recall@5 of 1.0, meaning that the expected source document appeared in the top five results for all five questions. Semantic chunking had the highest

average Top-1 cosine and the lowest average retrieval latency. Token chunking had the highest Mean@5 cosine. Sentence-window chunking produced the largest number of indexed chunks and had the highest retrieval latency.

### Confidently Scored Retrieval That Did Not Contain the Answer

```
Question: How long after an alleged housing discrimination incident occurs or ends does a person have to file a complaint with HUD?

Rank: 1
Cosine similarity: 0.808375
Source: corpus/hw03/hud_fair_housing_booklet.pdf
Page: 18
Source excerpt: gton, DC 20410-2000. In Addition You May File a Private Lawsuit: You may file a private civil lawsuit with out first filing a complaint with HUD. You must file your lawsuit within two (2) years of the most recent date of alleged discriminatory action. If you do file a complaint with HUD and even if HUD dismisses your complaint, the Fair Housing Act gives you the right to file a private civil lawsuit against the respondent(s) in

Rank: 2
Cosine similarity: 0.749331
Source: corpus/hw03/hud_fair_housing_booklet.pdf
Page: 10
Source excerpt: plaint online, write a letter or telephone the HUD office nearest you. You have one year after the alleged discrimination occurred or ended to file a complaint with HUD, but you should file it as soon as possible. If You Think y our Rights Have Been Violated What to Tell HUD: • Your name and address • The name and address of the person your complain t is against (the respondent) • The address or other identification of the housi
```

Question q5 asked how long a person has to file a housing discrimination complaint with HUD.

The first Semantic result had a cosine similarity of 0.808375. However, the passage discussed the two-year deadline for filing a private civil lawsuit. It did not answer the question about the HUD complaint deadline.

The second result had a lower cosine similarity of 0.749331, but it contained the correct answer: a person has one year after the alleged discrimination occurred or ended to file a complaint with HUD.

The first result received a high score because it contained closely related terms about housing discrimination, HUD, filing, and legal deadlines. This shows that semantic similarity does not guarantee that a retrieved passage contains the correct answer.

### 2. Observations (1–2 short paragraphs):

- Discuss why one technique performed better on this query (e.g., sentence coherence, semantic boundary detection, token-budget alignment, context carried via sentence window).
- If the best technique differs across your optional extra queries, mention it.

Semantic chunking performed best overall for this corpus. It produced 441 chunks, which was the smallest index, and achieved the highest average Top-1 cosine of 0.737317. It also had the lowest average retrieval latency at 9.39 milliseconds. Grouping semantically related sentences helped keep parts of the same legal explanation together.

Token chunking was also competitive and achieved the highest Mean@5 cosine of 0.686432. Sentence-window chunking preserved nearby context, but it created 3,797 indexed nodes and increased average retrieval latency to 36.61 milliseconds. Its additional context did not produce better overall measurements for this corpus.

### 3. Your conclusion (2–5 sentences):

- State which technique you judge best for this corpus and why supported by your Measurements.

I consider Semantic chunking the best technique for this corpus. It produced the smallest index, achieved the highest average Top-1 cosine, and had the lowest average retrieval latency. Token chunking was also effective, but Sentence-window chunking required a much larger index without improving the overall retrieval measurements.

### Generative AI Use

```
Generative AI Use and Verification
```

```
1. What did I use an AI assistant for?
```

```
I used an AI assistant to help me explain the LlamaIndex components, organize the experiment code, and troubleshoot errors.
```

```
I created and edited the files in Cursor, chose and checked the corpus documents, prepared the five questions, ran the commands in my local environment, reviewed the output, and saved the screenshots and result files.
```

```
2. What AI output was incorrect or needed verification?
```

```
The AI assistant incorrectly advised me to calculate Top-1 cosine using
'cosine_values[0]'.
```

```
That calculation selected the cosine similarity of the first vector-store result.
However, the assignment defines Top-1 cosine as the highest cosine similarity among
the top-k results.
```

```
3. How did I detect the problem?
```

```
I checked the metric definition in the original HW3 instructions. The assignment
states that Top-1 cosine should be the highest similarity among the top-k results for
each technique.
```

```
The difference was visible in the sentence-window results because the vector store
ranked the center sentence, while my independent cosine calculation used the expanded
sentence window.
```

```
4. What did I change?
```

I restored the calculation to:

```
`max(cosine_values)`
```

I then reran the metrics script and regenerated the summary CSV, JSON, and per-query results.

After the correction, the average Sentence-window Top-1 cosine changed from 0.593357 to 0.723145. This value now follows the metric definition in the assignment.

## Final Verification

```
def main() -> None:
 run_check("part1_application", check_application)
 run_check("part2_corpus", check_corpus)
 run_check("corpus_manifest", check_manifest)
 run_check("evaluation_questions", check_questions)
 run_check("questions_precommitted", check_precommit_order)
 run_check("retrieval_outputs", check_retrieval_outputs)
 run_check("metrics_and_analysis", check_metrics)
 run_check("screenshots", check_screenshots)

 passed_count = sum(check["passed"] for check in checks)
 overall_passed = passed_count == len(checks)
```

The self-check script verifies the authentication workflow, corpus, manifest, evaluation questions, Git commit order, retrieval outputs, metrics files, analysis, and required screenshots.

```
((.venv)) carol@Carols-MacBook-Air data260-2974 % python -m py_compile code/verify_hw3.py
((.venv)) carol@Carols-MacBook-Air data260-2974 % python code/verify_hw3.py
[PASS] part1_application
[PASS] part2_corpus
[PASS] corpus_manifest
[PASS] evaluation_questions
[PASS] questions_precommitted
[PASS] retrieval_outputs
[PASS] metrics_and_analysis
[PASS] screenshots

Checks passed: 8/8
Overall passed: True
Saved verification: /Users/carol/Documents/data260-2974/reports/hw03/verification.json
((.venv)) carol@Carols-MacBook-Air data260-2974 %
```